

Final Project - Starbucks

Team Members:

Braeden Villella, Cameron Neidrich, Charlie Krzyminski, Thurman Schreel, and Carson
Porter

April 30th, 2026

Course: ISA 345

Section: B

Instructor: Maliha Alam

1. Project Overview

Business Setting

In a world where everything is moving online, Starbucks wants to stay competitive in the coffee industry, so it is looking to transition from paper-based data to an online database that will make it easier to add and extract data for business decisions. We are looking to design a database that will efficiently support Starbucks's operations.

Business Process

The database we are developing will include customer information that connects to orders at different locations. Each location will have one boss and employees who provide the customer with their order using predetermined recipes. The ingredients for these products are supplied by suppliers.

Activities

The database will represent the activities of adding a new product and recipe to the menu for Starbucks, adding employees to locations, adding customers when they order, and adding locations when a new store is desired.

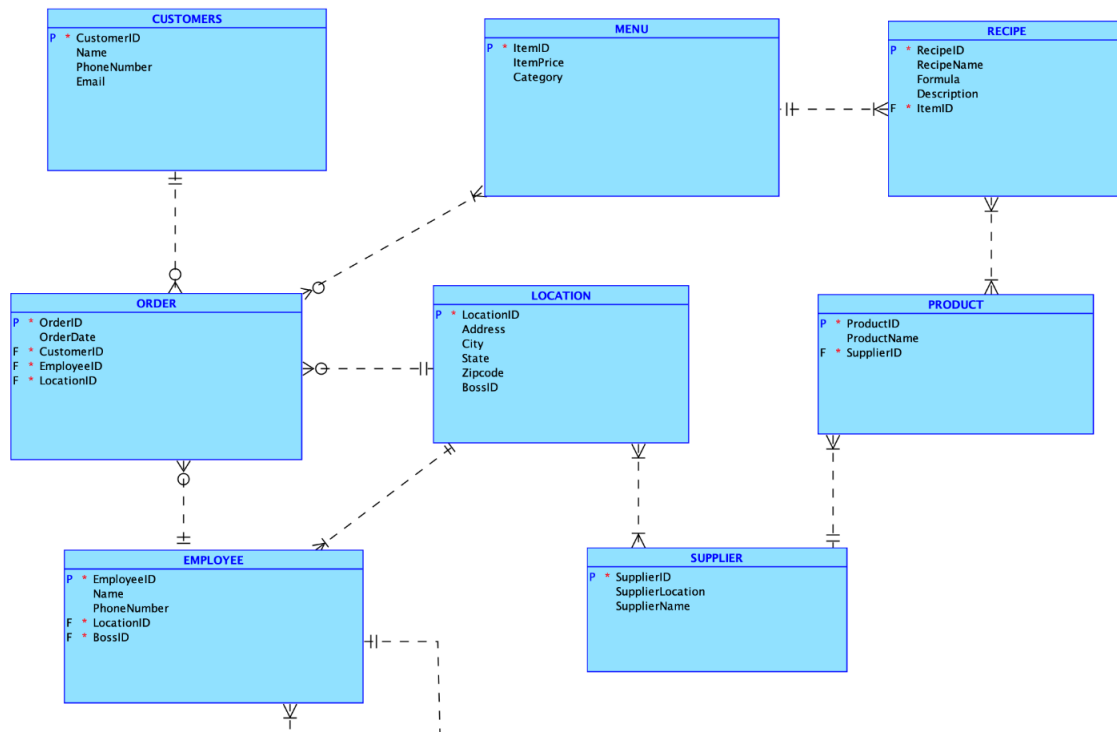
Operational Transactions

An operational transaction associated with our database includes when an order is placed. When an order is placed, the customer who ordered is included, the location where the order was placed is included, and the employee who prepared the order is included. Another operational transaction example would be hiring another employee for the company. This would be adding another employee to their table and would include the location they are working at, their personal information, and their supervisor's ID. A final example of an operational transaction would be adding a new supplier for the company. This would include adding a new location where the supplier resides and then updating what products the new supplier supplies to Starbucks.

2. Database Design and Implementation

Our database design will start as a logical model to illustrate the entities, attributes, and relationships present between entities in the database. Below, we inserted an image of the created logical model.

Logical Model:



Entity Descriptions

The entities that are in our database read as follows: Customer, Menu, Location, Orders, Employee, Recipe, Product, Supplier. For each of these entities, we created a description of each entity and its attributes. Some of these entities will create associative tables when they are converted to a physical model

Customer

The Customer table provides a unique identifier for each customer, the CustomerID. Each customer has a name, a unique email address, and a unique phone number. Each customer can make many orders, but each order only belongs to one customer.

- CustomerID - PK
- CustomerName - Can't be null
- PhoneNumber - Unique, can't be null
- Email - Unique, can't be null

Orders

The Order table has a unique identifier for each order, OrderID. Each order contains the date that the order was placed, the location that the order was placed, the customer who placed the order, and the employee who prepared the order.

- OrderID - PK
- OrderDate - Can't be null
- LocationID - FK, can't be null
- CustomerID - FK, can't be null
- EmployeeID - FK, can't be null

Employee

The Employee table has a unique identifier for each employee, EmployeeID. Each employee has a name, a phone number unique to the employee, a LocationID for their store, and a BossID.

- EmployeeID - PK
- EmployeeName - Can't be null
- Phone Number - Unique, can't be null
- BossID - FK
- WorkLocation - FK

Location

The Location table has a unique identifier for each store location, LocationID. Each location has an address, the city where it is located, the state it is located in, and the zip code it is located in, and the boss of the location.

- LocationID - PK
- Address - Can't be null
- City - Can't be null
- State - Can't be null
- Zipcode - Can't be null
- BossID - FK, unique, can't be null

Menu

The Menu item is the table where all of the different items for sale are presented; the items are uniquely identified with ItemID. Each item's name is displayed, the price of the item, and the category of the item (e.g. hot drink, food, etc.).

- ItemID - PK
- ItemName - Can't be null
- ItemPrice - Can't be null
- Category - Can't be null

Recipe

The recipes table has a unique identifier for each recipe, RecipeID. There is also a formula for steps to make the recipe, a description of what the recipe is, and the corresponding ItemID the recipe is for.

- RecipeID - PK
- Formula - Can't be null
- Description - Can't be null
- ItemID - FK, can't be null

Product

The product table has a unique identifier for each product, ProductID. For each product, there is a ProductName that describes the product. There is also the SupplierID which links to the SUPPLIER table. One product is supplied by one supplier, but with this, a supplier can provide many different products.

- ProductID - PK
- ProductName - Can't be null
- SupplierID - FK, can't be null

Supplier

The Supplier is to specify what supplier ingredients are sourced from. The Supplier table has a unique identifier for each supplier, SupplierID. For each supplier, there is the SupplierLocation, which is the region the supplier is located. There is also a SupplierName which is the company name of the supplier.

- SupplierID - PK
- SupplierLocation - Can't be null
- SupplierName - Can't be null

Location_Supplier

The Location_Supplier table allows for a many-to-many relationship between Location and Supplier. This is because one location can have multiple suppliers for their products, while a supplier can also supply multiple locations. This table contains a composite primary key of LocationID to specify location supplied, and SupplierID to determine the supplier. The DeliveryDate attribute is also in the composite primary key to allow for a supplier to have multiple historical deliveries to the same location. This attribute describes the date the location received a shipment.

- LocationID - PK, FK
- SupplierID - PK, FK
- DeliveryDate - PK

Product_Recipe

The Product_Recipe table allows for the many-to-many relationship between Product and Recipe, where an ingredient can be in multiple recipes, while recipes can also have multiple ingredients in them. A composite primary key of RecipeID, which specifies the recipe, and ProductID, which specifies the ingredient used. To know the unit of measurement for the ingredient (e.g. tablespoon, cup) we have a Unit attribute. The Quantity attribute determines how many units to include in the recipe.

- RecipeID - PK, FK

- ProductID - PK, FK
- Unit - Can't be null
- Quantity - Can't be null

Lineitem

The Lineitem table allows for a many-to-many relationship for an order to have multiple items, as well as an item to be in many orders. This table contains a composite primary key of OrderID to specify the order, and ItemID to determine the items ordered.

- OrderID - PK, FK
- ItemID - PK, FK
- ItemQuantity - Can't be null

Business Rules and Cardinalities

There were various business rules we had to incorporate into our database that followed real-life scenarios and ensured the database followed regular operations. Ensuring all of our entities and attributes were subject to these business rules made our database more consistent and reliable. The business rules we incorporated into our database are as follows:

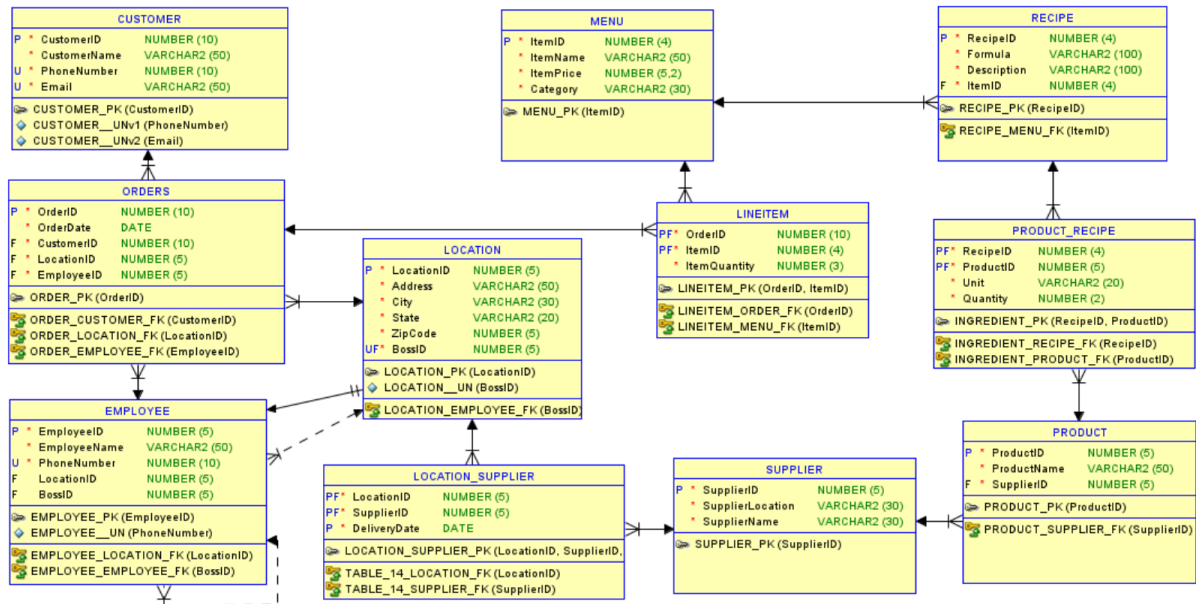
- Each customer can place many orders
- Each order can only be from one customer
- Each order is fulfilled by one employee
- Each employee belongs to a location
- Each location can have multiple employees
- Each location has a boss assigned to it
- Each boss is also an employee of the company
- Each order can have multiple items
- Each item has a recipe
- Each recipe can be made from multiple products
- Products can be used for multiple recipes
- Each product comes from a supplier
- Suppliers can supply multiple products
- Each supplier has their own location

Normalization

Our model originally had ItemName in ORDERS, but this led to multiple records if an order contained multiple items. To achieve 3NF, ORDERS now stores the order-level

data. LINEITEM now stores the item entry data, allowing for normalization of orders with multiple items. MENU contains information about each item, which LINEITEM connects MENU and ORDERS.

Physical Model



3. Operational SQL

Here shows the managerial value of our given database through five complete queries, plus four supporting database objects (INDEX, VIEW, PROCEDURE, TRIGGER). Each of our queries is linked to a real business decision Starbucks managers would face.

Query 1 - Subquery

This query identifies the customers that have reported to spend more on Starbucks orders compared to the average customer spend. The subquery does this by comparing each customer's total they spend to the average spend across all of the customers. Using this data, management can locate and evaluate the high value customers for things like perks, loyalty rewards, and targeted retention techniques.

```
SELECT CUSTOMER.CustomerID, CUSTOMER.CustomerName,
SUM(ORDERS.OrderCost) AS TotalSpent
FROM CUSTOMER
INNER JOIN ORDERS ON CUSTOMER.CustomerID = ORDERS.CustomerID
GROUP BY CUSTOMER.CustomerID, CUSTOMER.CustomerName
HAVING SUM(ORDERS.OrderCost) > (
    SELECT AVG(CustomerTotal)
    FROM (SELECT SUM(OrderCost) AS CustomerTotal
    FROM ORDERS
    GROUP BY CustomerID))
ORDER BY TotalSpent DESC;
```

	CUSTOMERID	CUSTOMERNAME	TOTALSPENT
1	4	Diana Prince	31.25
2	1	Alice Johnson	30.75
3	6	Fiona Green	26.25
4	2	Bob Smith	25

Query 2 - Inner Join

This query we created identifies what the most popular menu items are. It does so by counting the number of how many times that each of the Starbucks items have been ordered. The purpose for joining the Menu and Orders tables is so that the management can see which items are responsible for driving the most volume information critical for inventory planning, promotional decisions, and seasonal menu rotations.

```
SELECT MENU.ItemID, MENU.ItemName, MENU.Category,  
SUM(LINEITEM.ItemQuantity) AS TotalUnitsSold  
FROM MENU  
JOIN LINEITEM ON MENU.ItemID = LINEITEM.ItemID  
JOIN ORDERS ON LINEITEM.OrderID = ORDERS.OrderID  
GROUP BY MENU.ItemID, MENU.ItemName, MENU.Category  
ORDER BY TotalUnitsSold DESC;
```

	ITEMID	ITEMNAME	CATEGORY	TOTALUNITSSOLD
1	101	Latte	Drink	10
2	103	Espresso	Drink	10
3	107	Chocolate Croissant	Food	9
4	104	Mocha	Drink	9
5	106	Blueberry Muffin	Food	9
6	102	Cappuccino	Drink	6
7	105	Americano	Drink	4

Query 3 - Outer Join

This query works to list every menu item that we have with the number of times each has been ordered. This also shows items that have never been ordered. Using a LEFT OUTER JOIN ensures that even the unordered menu items appear in our result. This is the information managers need and want to see in order to identify underperforming products.

```
SELECT MENU.ItemID, MENU.ItemName, MENU.Category, MENU.ItemPrice,  
COUNT(LINEITEM.OrderID) AS TimesOrdered  
FROM MENU  
LEFT OUTER JOIN LINEITEM ON MENU.ItemID = LINEITEM.ItemID  
LEFT OUTER JOIN ORDERS ON LINEITEM.OrderID = ORDERS.OrderID
```

GROUP BY MENU.ItemID, MENU.ItemName, MENU.Category, MENU.ItemPrice
ORDER BY TimesOrdered ASC;

	ITEMID	ITEMNAME	CATEGORY	ITEMPRICE	TIMESORDERED
1	108	Bagel	Food	2.5	0
2	105	Americano	Drink	3.5	4
3	102	Cappuccino	Drink	4.25	6
4	104	Mocha	Drink	4.75	8
5	107	Chocolate Croissant	Food	3.25	9
6	106	Blueberry Muffin	Food	2.75	9
7	103	Espresso	Drink	3	10
8	101	Latte	Drink	4.5	10

Query 4 - GROUP BY without HAVING

This query works to count the number of orders that have been placed at each of the Starbucks locations. This provides our management team with a clear, outlined view of which of the stores are producing the highest-traffic and which ones are underperforming. Using the results, we are able to begin creating decisions about staffing, supply allocation, and even whether certain locations should be expanded or closed. By aggregating order counts per location, regional managers are able to look this and quickly identify which of our Starbucks stores need more support and which of our Starbucks stores are operating at it's peak efficiency.

```
SELECT LOCATION.LocationID, LOCATION.City, LOCATION.State,
COUNT(ORDERS.OrderID) AS TotalOrders
FROM LOCATION
INNER JOIN ORDERS ON LOCATION.LocationID = ORDERS.LocationID
GROUP BY LOCATION.LocationID, LOCATION.City, LOCATION.State
ORDER BY TotalOrders DESC;
```

	LOCATIONID	CITY	STATE	TOTALORDERS
1	1	Oxford	OH	14
2	3	Nashville	TN	9
3	2	Los Angeles	CA	7

Query 5 - GROUP BY with HAVING

This query was created to identify Starbucks locations that receive fewer than 10 total orders. This will help show what locations are underperforming and lacking the traffic needed to properly run the store. By singling these stores out, we are able to help management create better-informed decisions to better promote and strategize for the store's location. This will hopefully result in better performance and an increase in sales and revenue for the company. Using the HAVING clause helps us better filter aggregated results after counting the total number of orders per location.

```
SELECT LOCATION.LocationID, LOCATION.City, LOCATION.State,  
COUNT(ORDERS.OrderID) AS TotalOrders  
FROM LOCATION  
INNER JOIN ORDERS ON LOCATION.LocationID = ORDERS.LocationID  
GROUP BY LOCATION.LocationID, LOCATION.City, LOCATION.State  
HAVING COUNT(ORDERS.OrderID) < 10  
ORDER BY TotalOrders DESC;
```

	LOCATIONID	CITY	STATE	TOTALORDERS
1	3	Nashville	TN	9
2	2	Los Angeles	CA	7

Index

This Query helps us quickly identify how many orders a store is receiving. This way, management is able to quickly determine issues or successes occurring at stores to efficiently implement new ideas.

```
CREATE INDEX OrdersAtLocationsIndex ON ORDERS(LocationID);
```

View

The view that we have created shows a customer view summary of their order history without showing the internal employee assignments or location level data, which of course customers do not need to see. This can be used in a number of different ways such as a mobile app or web interface, to display a customer's order history safely.

```

CREATE VIEW CustomerOrderHistory AS
SELECT CUSTOMER.CustomerID,
       CUSTOMER.CustomerName,
       ORDERS.OrderID,
       ORDERS.OrderDate,
       MENU.Category,
       ORDERS.OrderCost
FROM CUSTOMER
INNER JOIN ORDERS ON CUSTOMER.CustomerID = ORDERS.CustomerID
INNER JOIN LINEITEM ON ORDERS.OrderID = LINEITEM.OrderID
INNER JOIN MENU ON LINEITEM.ItemID = MENU.ItemID;

```

	↕ CUSTO...	↕ CUSTOMERNAME	↕ ORDERID	↕ ORDERDATE	↕ CATEGORY	↕ ORDERCOST
1	1	Alice Johnson	601	10-APR-26	Drink	7.25
2	6	Fiona Green	606	15-APR-26	Drink	6.25
3	7	George White	607	16-APR-26	Drink	7.5
4	4	Diana Prince	612	18-APR-26	Drink	8.25
5	7	George White	615	20-APR-26	Drink	7.75
6	1	Alice Johnson	617	21-APR-26	Drink	4.5
7	4	Diana Prince	620	22-APR-26	Drink	7
8	8	Hannah Black	624	24-APR-26	Drink	7.8
9	3	Charlie Brown	627	26-APR-26	Drink	6.75

Procedure

The procedure was created to automatically insert values into the ORDER and LINEITEM tables when an order is made using the CALL function. It automatically calculates the total cost of the order based on the quantity of an item ordered. This is extremely helpful so that every time a customer places an order, the LINEITEM table entry is automatically generated through this procedure, so it keeps the data flowing as it should. The procedure also automatically inserts the current day, so the date of the order does not have to be manually entered and will automatically update over time.

```

CREATE PROCEDURE PlaceOrder (p_OrderID NUMBER, p_CustomerID NUMBER,
p_LocationID NUMBER, p_EmployeeID NUMBER, p_ItemID NUMBER, p_Quantity
NUMBER )
IS
    v_price NUMBER;
BEGIN
    SELECT ItemPrice INTO v_price
    FROM MENU
    WHERE ItemID = p_ItemID;

    INSERT INTO ORDERS
    VALUES (p_OrderID, SYSDATE, v_price * p_Quantity, p_CustomerID,
p_LocationID, p_EmployeeID);

    INSERT INTO LINEITEM
    VALUES (p_OrderID, p_ItemID, p_Quantity);
END;
/

```

Before Procedure: (ORDERS table and LINEITEM table respectively)

- Code used for procedure test in the Appendix

ORDERID	ORDERDATE	ORDERC...	CUSTOME...	LOCATIO...	EMPLOYEE...
---------	-----------	-----------	------------	------------	-------------

ORDE...	ITEMID	ITEMQUA...
---------	--------	------------

After Procedure (ORDERS table and LINEITEM table respectively)

	ORDERID	ORDERDATE	ORDERCOST	CUSTOMERID	LOCATIONID	EMPLOYEEID
1	7001	30-APR-26	9	1	1	304
2	7002	30-APR-26	4.25	2	1	305
3	7003	30-APR-26	14.25	3	2	306
4	7004	30-APR-26	3	4	3	308

	ORDERID	ITEMID	ITEMQUANTITY
1	7001	101	2
2	7002	102	1
3	7003	104	3
4	7004	103	1

Trigger

The trigger created works to automatically flag the Customer table when a customer reaches a high spending. When done with every insert into Orders, the trigger that we created for Starbucks works to check whether the customer's cumulative spending has passed the defined VIP cutoff and then proceeds to update their status accordingly.

```
CREATE TRIGGER CheckVIPStatus
AFTER INSERT ON ORDERS
BEGIN
    UPDATE CUSTOMER
    SET VIPStatus = 'Yes'
    WHERE CustomerID IN (
        SELECT CustomerID
        FROM ORDERS
        GROUP BY CustomerID
        HAVING SUM(OrderCost) > 500
    );
END;
/
```

Before the customer spent over \$500:

- Code for trigger test in the Appendix

	CUSTOMERID	VIPSTATUS
1	1	No

After the customer spent over \$500:

	 CUSTOMERID	 VIPSTATUS
1	1	Yes

4. Conclusion & Lessons Learned

This database works better than the previous paper database that was in place because of the ways it can be used to access all of the information instantly, instead of sorting through paper copies. If you were trying to see which store location was the busiest, you could use a query to sort each location by the number of orders to identify the busiest or most popular locations. We display this business problem and solution in query 4 so managers can see which locations are performing best and worst. If managers wanted to know which products were the most popular to try and identify consumer trends, they could sort all of the orders and identify which items were ordered the most. We accomplished this in query 3. This information could help managers make better decisions in many aspects of the company, like identifying which ingredients are used the most from those popular menu items, which then gives insight into which suppliers to order more from. Query 3 also displays the items that have never been ordered. This consumer data can be used by upper management to shape which products they want to create and promote in the future, or to identify which types of items have not worked well in the past, so they can avoid them.

Some of our lessons learned from the project were that there are many moving parts to a company like Starbucks, so you have to make sure you are always including the main functions of the company's operations. This also showed us just how important it is to have proper database design. Without a proper design, there can be many issues, such as inaccuracies, inefficiencies, and data redundancy that not only affect the system itself but can also affect various levels of management. This leads to another lesson, which is to always keep all the moving parts of Starbucks in mind while creating a database and queries. This is critical as the results provided by your system will guide decisions made by the company at all levels. Overall, this project was the perfect way for us to learn just how important it is to design databases with both structure and usability in mind to effectively support real-world business decisions.

5. Appendices

Implementation/Table Creation Notes

```
DROP TABLE MENU CASCADE CONSTRAINTS;  
DROP TABLE RECIPE CASCADE CONSTRAINTS;  
DROP TABLE CUSTOMER CASCADE CONSTRAINTS;  
DROP TABLE ORDERS CASCADE CONSTRAINTS;  
DROP TABLE EMPLOYEE CASCADE CONSTRAINTS;  
DROP TABLE LOCATION CASCADE CONSTRAINTS;  
DROP TABLE LINEITEM CASCADE CONSTRAINTS;  
DROP TABLE SUPPLIER CASCADE CONSTRAINTS;  
DROP TABLE LOCATION_SUPPLIER CASCADE CONSTRAINTS;  
DROP TABLE PRODUCT CASCADE CONSTRAINTS;  
DROP TABLE PRODUCT_RECIPE CASCADE CONSTRAINTS;
```

```
CREATE TABLE CUSTOMER (  
    CustomerID NUMBER(10) CONSTRAINT CustomerIDPK PRIMARY KEY,  
    CustomerName VARCHAR2(50) CONSTRAINT CustomerNameNotNull NOT NULL,  
    CustomerPhoneNumber NUMBER(10) CONSTRAINT CustPhoneNumberNotNull NOT NULL,  
    Email VARCHAR2(50) CONSTRAINT EmailNotNull NOT NULL,  
    CONSTRAINT CustPhoneNumberUnique UNIQUE (CustomerPhoneNumber),  
    CONSTRAINT EmailUnique UNIQUE (Email)  
);  
ALTER TABLE CUSTOMER ADD VIPStatus VARCHAR2(3);
```

```
CREATE TABLE MENU (  
    ItemID NUMBER(5) CONSTRAINT ItemIDPK PRIMARY KEY,  
    ItemName VARCHAR2(50) CONSTRAINT ItemNameNotNull NOT NULL,  
    ItemPrice NUMBER(5,2) CONSTRAINT ItemPriceNotNull NOT NULL,  
    Category VARCHAR2(30) CONSTRAINT CategoryNotNull NOT NULL  
);
```

```
CREATE TABLE LOCATION (  
    LocationID NUMBER(5) CONSTRAINT LocationIDPK PRIMARY KEY,  
    Address VARCHAR2(50) CONSTRAINT AddressNotNull NOT NULL,  
    City VARCHAR2(30) CONSTRAINT CityNotNull NOT NULL,  
    State VARCHAR2(20) CONSTRAINT StateNotNull NOT NULL,  
    ZipCode NUMBER(5) CONSTRAINT ZipCodeNotNull NOT NULL,  
    BossID NUMBER(5),  
    CONSTRAINT BossIDUnique UNIQUE (BossID)  
);
```

```
CREATE TABLE EMPLOYEE (  
    EmployeeID NUMBER(5) CONSTRAINT EmployeeIDPK PRIMARY KEY,  
    Name VARCHAR2(50) CONSTRAINT NameNotNull NOT NULL,  
    PhoneNumber NUMBER(10) CONSTRAINT PhoneNumberNotNull NOT NULL,  
    LocationID NUMBER(5) CONSTRAINT LocationIDNotNull NOT NULL,
```

```
BossID NUMBER(5),
CONSTRAINT PhoneNumberUnique UNIQUE (PhoneNumber),
CONSTRAINT EmployeeLocationFK FOREIGN KEY (LocationID) REFERENCES LOCATION (LocationID),
CONSTRAINT EmployeeEmployeeFK FOREIGN KEY (BossID) REFERENCES EMPLOYEE (EmployeeID)
);
```

```
ALTER TABLE LOCATION ADD CONSTRAINT LocationEmployeeFK FOREIGN KEY (BossID)
REFERENCES EMPLOYEE(EmployeeID);
```

```
CREATE TABLE ORDERS (
    ORDERID NUMBER(10) CONSTRAINT OrderIDPK PRIMARY KEY,
    OrderDate DATE CONSTRAINT OrderDateNotNull NOT NULL,
    OrderCost NUMBER(6,2) CONSTRAINT OrderCostNotNull NOT NULL,
    CustomerID NUMBER(10) CONSTRAINT CustomerIDNotNull NOT NULL,
    LocationID NUMBER(5) CONSTRAINT LocIDNotNull NOT NULL,
    EmployeeID NUMBER(5) CONSTRAINT EmployeeIDNotNull NOT NULL,
    CONSTRAINT OrdersCustomerFK FOREIGN KEY (CustomerID) REFERENCES CUSTOMER (CustomerID),
    CONSTRAINT OrdersLocationFK FOREIGN KEY (LocationID) REFERENCES LOCATION (LocationID),
    CONSTRAINT OrdersEmployeeFK FOREIGN KEY (EmployeeID) REFERENCES EMPLOYEE (EmployeeID)
);
```

```
CREATE TABLE LINEITEM (
    OrderID NUMBER(10),
    ItemID NUMBER(5),
    ItemQuantity NUMBER(3) CONSTRAINT ItemQuantityNotNull NOT NULL,
    CONSTRAINT OrdersLineitemFK FOREIGN KEY (OrderID) REFERENCES ORDERS (OrderID),
    CONSTRAINT MenuLineitemFK FOREIGN KEY (ItemID) REFERENCES MENU (ItemID),
    CONSTRAINT LineitemPK PRIMARY KEY (OrderID, ItemID)
);
```

```
CREATE TABLE RECIPE (
    RecipeID NUMBER(4) CONSTRAINT RecipeIDPK PRIMARY KEY,
    Formula VARCHAR2(100) CONSTRAINT FormulaNotNull NOT NULL,
    Description VARCHAR2(100) CONSTRAINT DescriptionNotNull NOT NULL,
    ItemID NUMBER(5) CONSTRAINT ItemIDNotNull NOT NULL,
    CONSTRAINT MenuRecipeFK FOREIGN KEY (ItemID) REFERENCES MENU(ItemID)
);
```

```
CREATE TABLE SUPPLIER (
    SupplierID NUMBER(5) CONSTRAINT SupplierIDPK PRIMARY KEY,
    SupplierLocation VARCHAR2(30) CONSTRAINT SupplierLocationNotNull NOT NULL,
    SupplierName VARCHAR2(30) CONSTRAINT SupplierNameNotNull NOT NULL
);
```

```
CREATE TABLE LOCATION_SUPPLIER (
    LocationID NUMBER(5),
    SupplierID NUMBER(5),
    DeliveryDate DATE,
    CONSTRAINT LocationSupplierLocationFK FOREIGN KEY (LocationID) REFERENCES LOCATION
(LocationID),
```

```

CONSTRAINT LocationSupplierSupplierFK FOREIGN KEY (SupplierID) REFERENCES SUPPLIER
(SupplierID),
CONSTRAINT LocationSupplierPK PRIMARY KEY (LocationID, SupplierID, DeliveryDate)
);

```

```

CREATE TABLE PRODUCT (
ProductID NUMBER(5) CONSTRAINT ProductIDPK PRIMARY KEY,
ProductName VARCHAR2(50) CONSTRAINT ProductNameNotNull NOT NULL,
SupplierID NUMBER(5) CONSTRAINT SupplierIDNotNull NOT NULL,
CONSTRAINT SupplierProductFK FOREIGN KEY (SupplierID) REFERENCES SUPPLIER (SupplierID)
);

```

```

CREATE TABLE PRODUCT_RECIPE (
RecipeID NUMBER(4),
ProductID NUMBER(5),
Unit VARCHAR2(20) CONSTRAINT UnitNotNull NOT NULL,
Quantity NUMBER(5) CONSTRAINT QtyNotNull NOT NULL,
CONSTRAINT PRODUCT_RECIPEPK PRIMARY KEY (RecipeID, ProductID),
CONSTRAINT ProductRecipeRecipeFK FOREIGN KEY (RecipeID) REFERENCES RECIPE (RecipeID),
CONSTRAINT ProductRecipeProductFK FOREIGN KEY (ProductID) REFERENCES PRODUCT (ProductID)
);

```

Insert Statements:

Note: The following data inserts were created with the assistance of ChatGPT on 4/29/26.

```

INSERT INTO CUSTOMER VALUES (1, 'Alice Johnson', 5135551001, 'alice@email.com', 'No');
INSERT INTO CUSTOMER VALUES (2, 'Bob Smith', 5135551002, 'bob@email.com', 'No');
INSERT INTO CUSTOMER VALUES (3, 'Charlie Brown', 5135551003, 'charlie@email.com', 'No');
INSERT INTO CUSTOMER VALUES (4, 'Diana Prince', 5135551004, 'diana@email.com', 'No');
INSERT INTO CUSTOMER VALUES (5, 'Ethan Hunt', 5135551005, 'ethan@email.com', 'No');
INSERT INTO CUSTOMER VALUES (6, 'Fiona Green', 5135551006, 'fiona@email.com', 'No');
INSERT INTO CUSTOMER VALUES (7, 'George White', 5135551007, 'george@email.com', 'No');
INSERT INTO CUSTOMER VALUES (8, 'Hannah Black', 5135551008, 'hannah@email.com', 'No');

```

```

INSERT INTO MENU VALUES (101, 'Latte', 4.50, 'Drink');
INSERT INTO MENU VALUES (102, 'Cappuccino', 4.25, 'Drink');
INSERT INTO MENU VALUES (103, 'Espresso', 3.00, 'Drink');
INSERT INTO MENU VALUES (104, 'Mocha', 4.75, 'Drink');
INSERT INTO MENU VALUES (105, 'Americano', 3.50, 'Drink');
INSERT INTO MENU VALUES (106, 'Blueberry Muffin', 2.75, 'Food');
INSERT INTO MENU VALUES (107, 'Chocolate Croissant', 3.25, 'Food');
INSERT INTO MENU VALUES (108, 'Bagel', 2.50, 'Food');

```

```

INSERT INTO SUPPLIER VALUES (201, 'Cincinnati', 'Queen City Roasters');
INSERT INTO SUPPLIER VALUES (202, 'Columbus', 'Buckeye Bean Supply');
INSERT INTO SUPPLIER VALUES (203, 'Dayton', 'Wright Cup Distributors');
INSERT INTO SUPPLIER VALUES (204, 'Indianapolis', 'Indy Brew Provisions');

```

INSERT INTO LOCATION VALUES (1, '123 Main St', 'Oxford', 'OH', 45056, NULL);
INSERT INTO LOCATION VALUES (2, '456 Chocolate St', 'Los Angeles', 'CA', 9001, NULL);
INSERT INTO LOCATION VALUES (3, '789 Coffee Ave', 'Nashville', 'TN', 37011, NULL);

INSERT INTO EMPLOYEE VALUES (301, 'Sarah Manager', 5135552001, 1, NULL);
INSERT INTO EMPLOYEE VALUES (302, 'John Supervisor', 5135552002, 2, NULL);
INSERT INTO EMPLOYEE VALUES (303, 'Karen Lead', 5135552003, 3, NULL);

UPDATE LOCATION SET BossID = 301 WHERE LocationID = 1;
UPDATE LOCATION SET BossID = 302 WHERE LocationID = 2;
UPDATE LOCATION SET BossID = 303 WHERE LocationID = 3;

UPDATE EMPLOYEE SET LocationID = 1 WHERE EmployeeID = 301;
UPDATE EMPLOYEE SET LocationID = 2 WHERE EmployeeID = 302;
UPDATE EMPLOYEE SET LocationID = 3 WHERE EmployeeID = 303;

INSERT INTO EMPLOYEE VALUES (304, 'Emma Barista', 5135552004, 1, 301);
INSERT INTO EMPLOYEE VALUES (305, 'Liam Barista', 5135552005, 1, 301);
INSERT INTO EMPLOYEE VALUES (306, 'Olivia Cashier', 5135552006, 2, 302);
INSERT INTO EMPLOYEE VALUES (307, 'Noah Cashier', 5135552007, 2, 302);
INSERT INTO EMPLOYEE VALUES (308, 'Ava Barista', 5135552008, 3, 303);
INSERT INTO EMPLOYEE VALUES (309, 'Mason Barista', 5135552009, 3, 303);

INSERT INTO PRODUCT VALUES (401, 'Coffee Beans', 201);
INSERT INTO PRODUCT VALUES (402, 'Milk', 202);
INSERT INTO PRODUCT VALUES (403, 'Sugar', 203);
INSERT INTO PRODUCT VALUES (404, 'Chocolate Syrup', 204);
INSERT INTO PRODUCT VALUES (405, 'Flour', 201);
INSERT INTO PRODUCT VALUES (406, 'Blueberries', 202);
INSERT INTO PRODUCT VALUES (407, 'Butter', 203);

INSERT INTO RECIPE VALUES (501, 'Beans + Milk', 'Latte Recipe', 101);
INSERT INTO RECIPE VALUES (502, 'Beans + Foam Milk', 'Cappuccino Recipe', 102);
INSERT INTO RECIPE VALUES (503, 'Beans Only', 'Espresso Recipe', 103);
INSERT INTO RECIPE VALUES (504, 'Beans + Chocolate + Milk', 'Mocha Recipe', 104);
INSERT INTO RECIPE VALUES (505, 'Beans + Water', 'Americano Recipe', 105);
INSERT INTO RECIPE VALUES (506, 'Flour + Blueberries + Butter', 'Muffin Recipe', 106);

INSERT INTO PRODUCT_RECIPE VALUES (501, 401, 'grams', 10);
INSERT INTO PRODUCT_RECIPE VALUES (501, 402, 'ml', 200);
INSERT INTO PRODUCT_RECIPE VALUES (502, 401, 'grams', 10);
INSERT INTO PRODUCT_RECIPE VALUES (502, 402, 'ml', 150);
INSERT INTO PRODUCT_RECIPE VALUES (503, 401, 'grams', 8);
INSERT INTO PRODUCT_RECIPE VALUES (504, 401, 'grams', 10);
INSERT INTO PRODUCT_RECIPE VALUES (504, 402, 'ml', 150);
INSERT INTO PRODUCT_RECIPE VALUES (504, 404, 'ml', 50);
INSERT INTO PRODUCT_RECIPE VALUES (505, 401, 'grams', 8);

INSERT INTO PRODUCT_RECIPE VALUES (506, 405, 'grams', 50);
INSERT INTO PRODUCT_RECIPE VALUES (506, 406, 'grams', 20);
INSERT INTO PRODUCT_RECIPE VALUES (506, 407, 'grams', 10);

INSERT INTO LOCATION_SUPPLIER VALUES (1, 201, DATE '2026-04-01');
INSERT INTO LOCATION_SUPPLIER VALUES (1, 202, DATE '2026-04-02');
INSERT INTO LOCATION_SUPPLIER VALUES (2, 202, DATE '2026-04-03');
INSERT INTO LOCATION_SUPPLIER VALUES (2, 203, DATE '2026-04-04');
INSERT INTO LOCATION_SUPPLIER VALUES (3, 203, DATE '2026-04-05');
INSERT INTO LOCATION_SUPPLIER VALUES (3, 204, DATE '2026-04-06');

INSERT INTO ORDERS VALUES (601, DATE '2026-04-10', 7.25, 1, 1, 304);
INSERT INTO ORDERS VALUES (602, DATE '2026-04-11', 6.75, 2, 1, 305);
INSERT INTO ORDERS VALUES (603, DATE '2026-04-12', 5.50, 3, 2, 306);
INSERT INTO ORDERS VALUES (604, DATE '2026-04-13', 8.00, 4, 2, 307);
INSERT INTO ORDERS VALUES (605, DATE '2026-04-14', 4.50, 5, 3, 308);
INSERT INTO ORDERS VALUES (606, DATE '2026-04-15', 6.25, 6, 3, 309);
INSERT INTO ORDERS VALUES (607, DATE '2026-04-16', 7.50, 7, 1, 304);
INSERT INTO ORDERS VALUES (608, DATE '2026-04-16', 5.25, 8, 1, 305);
INSERT INTO ORDERS VALUES (609, DATE '2026-04-17', 9.00, 1, 2, 306);
INSERT INTO ORDERS VALUES (610, DATE '2026-04-17', 4.75, 2, 2, 307);
INSERT INTO ORDERS VALUES (611, DATE '2026-04-18', 6.50, 3, 3, 308);
INSERT INTO ORDERS VALUES (612, DATE '2026-04-18', 8.25, 4, 3, 309);
INSERT INTO ORDERS VALUES (613, DATE '2026-04-19', 3.00, 5, 1, 304);
INSERT INTO ORDERS VALUES (614, DATE '2026-04-19', 6.75, 6, 1, 305);
INSERT INTO ORDERS VALUES (615, DATE '2026-04-20', 7.75, 7, 2, 306);
INSERT INTO ORDERS VALUES (616, DATE '2026-04-20', 5.50, 8, 2, 307);
INSERT INTO ORDERS VALUES (617, DATE '2026-04-21', 4.50, 1, 3, 308);
INSERT INTO ORDERS VALUES (618, DATE '2026-04-21', 9.25, 2, 3, 309);
INSERT INTO ORDERS VALUES (619, DATE '2026-04-22', 6.00, 3, 1, 304);
INSERT INTO ORDERS VALUES (620, DATE '2026-04-22', 7.00, 4, 1, 305);
INSERT INTO ORDERS VALUES (621, DATE '2026-04-23', 8.50, 5, 2, 306);
INSERT INTO ORDERS VALUES (622, DATE '2026-04-23', 5.75, 6, 3, 307);
INSERT INTO ORDERS VALUES (623, DATE '2026-04-24', 6.25, 7, 3, 308);
INSERT INTO ORDERS VALUES (624, DATE '2026-04-24', 7.80, 8, 3, 309);
INSERT INTO ORDERS VALUES (625, DATE '2026-04-25', 10.00, 1, 1, 304);
INSERT INTO ORDERS VALUES (626, DATE '2026-04-25', 4.25, 2, 1, 305);
INSERT INTO ORDERS VALUES (627, DATE '2026-04-26', 6.75, 3, 1, 306);
INSERT INTO ORDERS VALUES (628, DATE '2026-04-26', 8.00, 4, 1, 307);
INSERT INTO ORDERS VALUES (629, DATE '2026-04-27', 5.25, 5, 1, 308);
INSERT INTO ORDERS VALUES (630, DATE '2026-04-27', 7.50, 6, 1, 309);

INSERT INTO LINEITEM VALUES (601, 101, 1);
INSERT INTO LINEITEM VALUES (601, 106, 1);
INSERT INTO LINEITEM VALUES (602, 102, 1);
INSERT INTO LINEITEM VALUES (602, 103, 1);
INSERT INTO LINEITEM VALUES (603, 103, 1);
INSERT INTO LINEITEM VALUES (603, 107, 1);
INSERT INTO LINEITEM VALUES (604, 104, 1);

INSERT INTO LINEITEM VALUES (604, 106, 1);
INSERT INTO LINEITEM VALUES (605, 105, 1);
INSERT INTO LINEITEM VALUES (606, 101, 1);
INSERT INTO LINEITEM VALUES (606, 107, 1);
INSERT INTO LINEITEM VALUES (607, 101, 1);
INSERT INTO LINEITEM VALUES (607, 107, 1);
INSERT INTO LINEITEM VALUES (608, 103, 1);
INSERT INTO LINEITEM VALUES (608, 104, 1);
INSERT INTO LINEITEM VALUES (609, 104, 1);
INSERT INTO LINEITEM VALUES (609, 106, 1);
INSERT INTO LINEITEM VALUES (610, 104, 1);
INSERT INTO LINEITEM VALUES (611, 102, 1);
INSERT INTO LINEITEM VALUES (611, 103, 1);
INSERT INTO LINEITEM VALUES (612, 101, 1);
INSERT INTO LINEITEM VALUES (612, 106, 1);
INSERT INTO LINEITEM VALUES (612, 103, 1);
INSERT INTO LINEITEM VALUES (613, 103, 1);
INSERT INTO LINEITEM VALUES (614, 102, 1);
INSERT INTO LINEITEM VALUES (614, 107, 1);
INSERT INTO LINEITEM VALUES (615, 101, 1);
INSERT INTO LINEITEM VALUES (615, 103, 1);
INSERT INTO LINEITEM VALUES (616, 105, 1);
INSERT INTO LINEITEM VALUES (616, 106, 1);
INSERT INTO LINEITEM VALUES (617, 101, 1);
INSERT INTO LINEITEM VALUES (618, 104, 1);
INSERT INTO LINEITEM VALUES (618, 107, 1);
INSERT INTO LINEITEM VALUES (619, 102, 1);
INSERT INTO LINEITEM VALUES (619, 103, 1);
INSERT INTO LINEITEM VALUES (620, 101, 1);
INSERT INTO LINEITEM VALUES (620, 106, 1);
INSERT INTO LINEITEM VALUES (621, 104, 1);
INSERT INTO LINEITEM VALUES (621, 107, 1);
INSERT INTO LINEITEM VALUES (622, 105, 1);
INSERT INTO LINEITEM VALUES (622, 103, 1);
INSERT INTO LINEITEM VALUES (623, 102, 1);
INSERT INTO LINEITEM VALUES (623, 106, 1);
INSERT INTO LINEITEM VALUES (624, 101, 1);
INSERT INTO LINEITEM VALUES (624, 107, 1);
INSERT INTO LINEITEM VALUES (625, 104, 2);
INSERT INTO LINEITEM VALUES (625, 106, 1);
INSERT INTO LINEITEM VALUES (626, 102, 1);
INSERT INTO LINEITEM VALUES (627, 101, 1);
INSERT INTO LINEITEM VALUES (627, 103, 1);
INSERT INTO LINEITEM VALUES (628, 104, 1);
INSERT INTO LINEITEM VALUES (628, 107, 1);
INSERT INTO LINEITEM VALUES (629, 105, 1);
INSERT INTO LINEITEM VALUES (629, 106, 1);
INSERT INTO LINEITEM VALUES (630, 101, 1);
INSERT INTO LINEITEM VALUES (630, 107, 1);

Procedure calls examples:

```
SELECT * FROM ORDERS  
WHERE OrderID >= 7001  
ORDER BY OrderID;
```

```
SELECT * FROM LINEITEM  
WHERE OrderID >= 7001  
ORDER BY OrderID;
```

```
CALL PlaceOrder(7001, 1, 1, 304, 101, 2);  
CALL PlaceOrder(7002, 2, 1, 305, 102, 1);  
CALL PlaceOrder(7003, 3, 2, 306, 104, 3);  
CALL PlaceOrder(7004, 4, 3, 308, 103, 1);
```

```
SELECT * FROM ORDERS  
WHERE OrderID >= 7001  
ORDER BY OrderID;
```

```
SELECT * FROM LINEITEM  
WHERE OrderID >= 7001  
ORDER BY OrderID;
```

Trigger test and examples:

```
SELECT CustomerID, VIPStatus  
FROM CUSTOMER  
WHERE CustomerID = 1;
```

```
INSERT INTO ORDERS VALUES (9001, DATE '2026-04-29', 300, 1, 1, 301);  
INSERT INTO ORDERS VALUES (9002, DATE '2026-04-29', 250, 1, 1, 302);
```

```
SELECT CustomerID, VIPStatus  
FROM CUSTOMER  
WHERE CustomerID = 1;
```